

Impedance Model

Second order model representation

If we consider that a simple mass M , damper B , spring K , system is to be able to model the impedance behaviour of the human arm along an axis $\vec{x}$, while interacting with an environment then, we can write:

$$(Ms^2 + Bs + K)X_e(p) = F_e(p),$$

with X the endpoint position of the arm, and F the force of the arm on the environment, both along $\vec{x}$.

```
clear all
syms Ms Bs Ks;
```

The state representation of the system is:

$$\dot{X}(t) = \mathbf{A}X(t) + Bu(t),$$

$$y(t) = \mathbf{C}X(t)$$

$$X(t) = \begin{bmatrix} x_e(t) \\ \dot{x}_e(t) \end{bmatrix}, u(t) = f_e(t)$$

$$\mathbf{A} = \begin{bmatrix} 0 & 1 \\ -Ks/Ms & -Bs/Ms \end{bmatrix}$$

$$\mathbf{A} = \begin{pmatrix} 0 & 1 \\ -\frac{Ks}{Ms} & -\frac{Bs}{Ms} \end{pmatrix}$$

$$\mathbf{B} = [0; 1/Ms]$$

$$\mathbf{B} = \begin{pmatrix} 0 \\ \frac{1}{Ms} \end{pmatrix}$$

$$\mathbf{C} = [1, 0] \text{ \% or } \mathbf{C} = [0, 1] \text{ if the velocity is considered as output}$$

$$\mathbf{C} = \begin{matrix} 1 \times 2 \\ 1 & 0 \end{matrix}$$

Discretization

Assuming that the discretization is made through a zero order hold, and a sampling period T :

$$X[k+1] = A_d X[k] + B_d u[k]$$

$$A_d = e^{A^T},$$

$$B_d = \left(\int_{\tau=0}^T e^{A\tau} d\tau \right) B = A^{-1}(A_d - I)B, \exists A^{-1}$$

```
dt = 1e-3; % 1ms
Ad = expm(A*dt) % ilaplace(inv(s*eye(2)-A),s,dt)
```

Ad =

$$\begin{pmatrix} \frac{\sigma_2 \sqrt{\sigma_3} - Bs \sigma_2 + Bs \sigma_1 + \sigma_1 \sqrt{\sigma_3}}{2 \sqrt{\sigma_3}} & -\frac{\sigma_1 \sigma_3 + Bs^2 \sigma_2 - \sigma_2 \sigma_3 - Bs^2 \sigma_1}{4 Ks \sqrt{\sigma_3}} \\ \frac{Ks \sigma_2 - Ks \sigma_1}{\sqrt{\sigma_3}} & \frac{Bs \sigma_2 + \sigma_2 \sqrt{\sigma_3} - Bs \sigma_1 + \sigma_1 \sqrt{\sigma_3}}{2 \sqrt{\sigma_3}} \end{pmatrix}$$

where

$$\sigma_1 = e^{-\frac{Bs - \sqrt{\sigma_3}}{2000 Ms}}$$

$$\sigma_2 = e^{-\frac{Bs + \sqrt{\sigma_3}}{2000 Ms}}$$

$$\sigma_3 = Bs^2 - 4 Ks Ms$$

```
Bd = A\ (Ad - eye(2))*B % if A is not a singular matrix
```

Bd =

$$\left(\begin{array}{c} -\frac{e^{-\sigma_2} e^{-\sigma_1} (Bs^3 e^{\sigma_2} - Bs^3 e^{\sigma_1} - Bs e^{\sigma_2} \sigma_3 + Bs e^{\sigma_1} \sigma_3 + 2 Bs Ks Ms e^{\sigma_1} + 2 Ks Ms e^{\sigma_1} \sqrt{\sigma_3} - 2 Bs Ks Ms}{4 Ks^2 Ms \sqrt{\sigma_3}} \\ \frac{e^{-\sigma_2} e^{-\sigma_1} (e^{\sigma_1} \sigma_3 + Bs^2 e^{\sigma_2} - e^{\sigma_2} \sigma_3 - Bs^2 e^{\sigma_1})}{4 Ks Ms \sqrt{\sigma_3}} \end{array} \right)$$

where

$$\sigma_1 = \frac{Bs - \sqrt{\sigma_3}}{2000 Ms}$$

$$\sigma_2 = \frac{Bs + \sqrt{\sigma_3}}{2000 Ms}$$

$$\sigma_3 = Bs^2 - 4 Ks Ms$$

$$C_d = C;$$

We can now express the transfer function that relates the input force f_e to the output position x_e using

$H(z) = C_d (zI - A_d)^{-1} B_d$, with I the identity matrix.

```
z = sym('z');
Hd = simplify(Cd*(z*eye(2) - Ad)^(-1)*Bd);
[n,d] = numden(Hd);
n = collect(n,z)
```

n =

$$\left(2 e^{\frac{Bs}{1000 \text{ Ms}}} \sigma_5 - \sigma_4 - \sigma_2 + \sigma_1 - \sigma_3\right) z + \sigma_2 - \sigma_4 + 2 \sigma_5 - \sigma_1 - \sigma_3$$

where

$$\sigma_1 = Bs e^{\frac{Bs - \sigma_5}{2000 \text{ Ms}}}$$

$$\sigma_2 = Bs e^{\frac{Bs + \sigma_5}{2000 \text{ Ms}}}$$

$$\sigma_3 = e^{\frac{Bs - \sigma_5}{2000 \text{ Ms}}} \sigma_5$$

$$\sigma_4 = e^{\frac{Bs + \sigma_5}{2000 \text{ Ms}}} \sigma_5$$

$$\sigma_5 = \sqrt{Bs^2 - 4 Ks \text{ Ms}}$$

```
d = collect(d,z)
```

d =

$$\left(2 Ks e^{\frac{Bs}{1000 \text{ Ms}}} \sigma_1\right) z^2 + \left(-2 \left(Ks e^{\frac{Bs + \sigma_1}{2000 \text{ Ms}}} + Ks e^{\frac{Bs - \sigma_1}{2000 \text{ Ms}}}\right) \sigma_1\right) z + 2 Ks \sigma_1$$

where

$$\sigma_1 = \sqrt{Bs^2 - 4 Ks \text{ Ms}}$$

```
b_coef = flip(coeffs(n,z)); % high order first
a_coef = flip(coeffs(d,z));
```

We know now that we have a transfer function of the form:

$$\frac{X_e(z)}{F_e(z)} = H_d(z) = K_H \frac{z + b_0}{z^2 + a_1 z + a_0} = \frac{b_1' z + b_0'}{a_2' z^2 + a_1' z + a_0'}$$

$$K_H = \frac{b_1'}{a_2'}, b_0 = \frac{b_0'}{b_1'}, a_1 = \frac{a_1'}{a_2'}, a_0 = \frac{a_0'}{a_2'}$$

```
Kh = b_coef(1)/a_coef(1);
b0 = b_coef(2)/b_coef(1);
a1 = a_coef(2)/a_coef(1);
a0 = a_coef(3)/a_coef(1);
```

Identification

The parameters a_1, a_0, b_0, K can be estimated using identification methodologies, with real dataset from human robot physical interactions.

Data generation

Real unperturbed data

```
load 'data_2020_Nov_17/data_without_impacts_2020_11_17.mat' 't' 'thetas' 'forces_unf'
addpath('utils') %
addpath('.././force_torque_sensor') % for force pre-processing
```

The data from the experiment is preprocessed to obtain a real shaped signal of the interaction force, that the robots applies on the environment $f_{r \rightarrow e}$.

```
exp_nb = randi([1 15],1,1); % chose a random experimental dataset
t = t{exp_nb} - t{exp_nb}(1); % t0 = 0s
f_tmp = forces_filtering(forces_unf{exp_nb}', forces_unf{exp_nb}', thetas{exp_nb}', t); % from
%
[b,a] = butter(2,50/(1/(2*dt)),'low'); % BW 2nd order low pass filter (cutoff freq. 50 Hz)
f_tmp = -f_tmp(3,:); % fz conversion from f(e->r) to f(r->e), robot force on the environment
f = filtfilt(b,a,f_tmp); % zero phase digital filtering
clear forces_unf thetas f_tmp % clear unnecessary data
```

Simulation of the perturbations

This real input is then injected in a simulated system with known impedance parameters:

```
input_force.signals.values = f;
input_force.time = t;
% impedance parameters
Mv = 0.2;
Bv = 10;
Kv = 50;
% perturbation introduced in simulation
ext_signal = 1;
pert_mag = 10;
pert_space = ceil(2.8/dt); % samples
pert_duration = 0.030/dt; % samples
% perturbation filter
xi = sqrt(2)/2;
w0 = 40*pi*1;
K_f = 3e3;
t_max = t(end);
out = sim('Impedance_env_simulation/KBM_sim',t_max);
```

The results from the simulation are collected:

```
fz = out.force.data;
z = out.position.data;
t = out.force.Time;
```

```
% get the rising edges indexes of the perturbations
pert_idx = find(diff(out.perturbations.data) > 0)';
pert_val = pert_mag.*ones(size(pert_idx));
```

Virtual trajectories estimation

The data is then processed to extract the differential force and position trajectories between the virtual and the actual trajectories:

```
% PARAMETERS
idx_wndw_virt_traj = ceil(0.200/dt); % 200ms (position)
idx_wndw_virt_f_traj = ceil(0.065/dt); % 65ms (force)
idx_wndw_imp_eval = ceil(0.200/dt); % 200ms
idx_delay = ceil(0.000/dt); % 0ms
idx_window = max(idx_wndw_imp_eval, idx_wndw_virt_traj);
nb_param = 3; % K B M

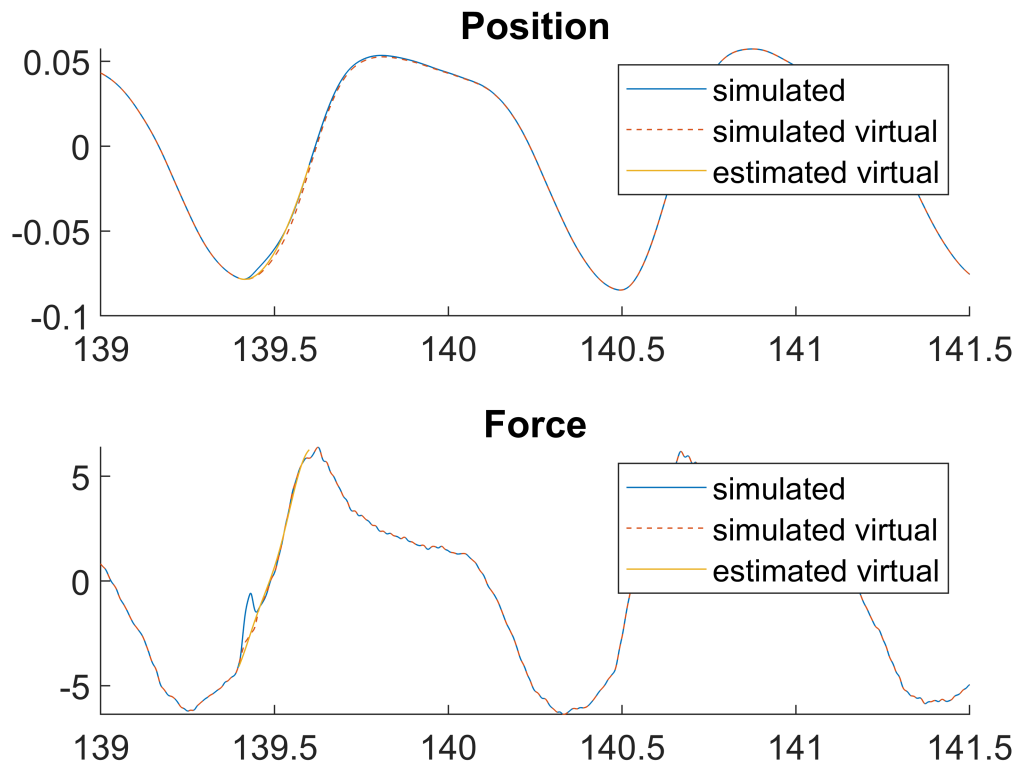
% DATA PRE-PROCESSING
delta_z = DIFF_TRAJECT(idx_window, idx_wndw_virt_traj, z, t, pert_idx, pert_val, idx_delay);
delta_fz = DIFF_TRAJECT(idx_window, idx_wndw_virt_f_traj, fz, t, pert_idx, pert_val, idx_delay);

delta_z.computeDiffTraject('VirtTrajMethod', 'spline');
delta_fz.computeDiffTraject('VirtTrajMethod', 'filterPlus'); % this step might take few seconds
delta_z.computeDerivatives();
```

Evaluation of the virtual trajectories

The produced methodology can be compared to the actual - *simulated* - virtual trajectories:

```
% real virtual trajectories
fz0 = out.virt_force.data;
z0 = out.virt_position.data;
%
figure('DefaultAxesFontSize',14)
subplot(2,1,1)
title('Position')
hold on
plot(t,z)
plot(t,z0, '--')
plot(delta_z.t_traject, delta_z.virt_traject, 'Color', [0.9290, 0.6940, 0.1250])
xlim([139 141.5])
legend('simulated', 'simulated virtual', 'estimated virtual')
subplot(2,1,2)
title('Force')
hold on
plot(t,fz)
plot(t,fz0, '--')
plot(delta_fz.t_traject, delta_fz.virt_traject, 'Color', [0.9290, 0.6940, 0.1250])
xlim([139 141.5])
legend('simulated', 'simulated virtual', 'estimated virtual')
```



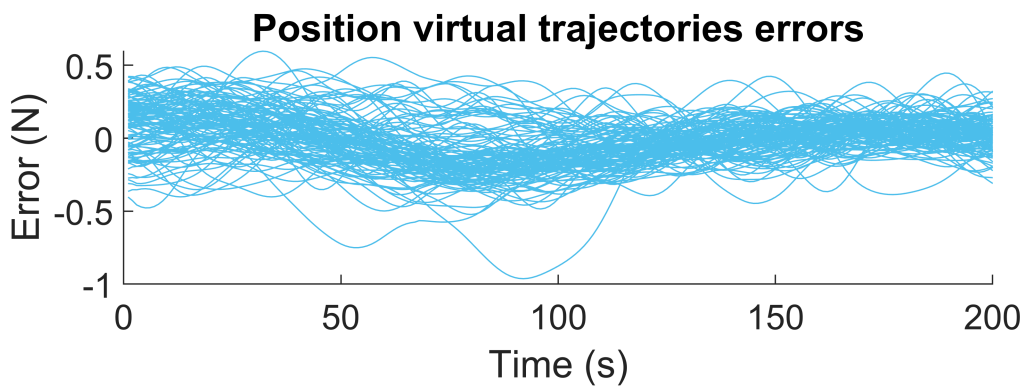
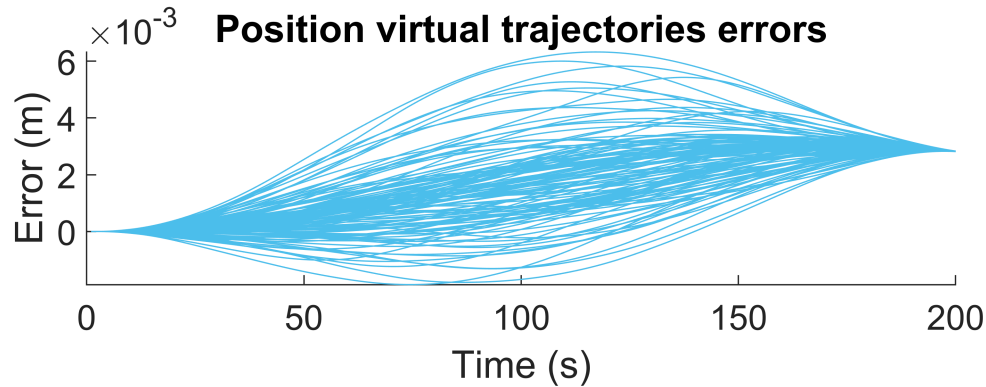
```
% Actual simulated virtual trajectories
delta_z_sim = copyObj(delta_z);
delta_fz_sim = copyObj(delta_fz);
delay_wdw = delta_z.delay;
estim_wdw = delta_z.estim_window;
for i = 1:delta_z_sim.nb_trajet
    pert_interval = delta_z_sim.pert_ind(i)+(-2:estim_wdw+1)+delay_wdw;
    delta_z_sim.virt_trajet(:,i) = z0(pert_interval);
    delta_fz_sim.virt_trajet(:,i) = fz0(pert_interval);
end
delta_z_sim.computeDiffTraject('VirtTrajMethod', 'manual');
delta_fz_sim.computeDiffTraject('VirtTrajMethod', 'manual');
% The result should be the same in this ideal scenario
```

```
%
figure('DefaultAxesFontSize',14)
subplot(2,1,1)
hold on
plot(delta_z.virt_trajet-delta_z_sim.virt_trajet, 'Color', [0.3010, 0.7450, 0.9330, 0.4])
title('Position virtual trajectories errors')
xlabel('Time (s)')
ylabel('Error (m)')
xlim([0, idx_wndw_imp_eval])
subplot(2,1,2)
hold on
plot(delta_fz.virt_trajet-delta_fz_sim.virt_trajet, 'Color', [0.3010, 0.7450, 0.9330, 0.4])
```

```

title('Position virtual trajectories errors')
xlabel('Time (s)')
ylabel('Error (N)')
xlim([0, idx_wndw_imp_eval])

```



Discrete time identification algorithms

We define two polynomials A_k and B_k such as: %% TODO rewrite this expression in accordance to the new representation using the transfer function

$$y(t) + a_1 y(t-1) + a_0 y(t-2) = b_0 u(t) + e(t)$$

$$A_k(q)y(t) = B_k(q)u(t) + e(t),$$

with q the delay operator.

```

% Data formatting for identification
for i = delta_z.nb_trajet:-1:1
    data{i} = iddata(delta_z.diff_trajet(:,i), ...
        delta_fz.diff_trajet(:,i),dt);
    data{i}.TimeUnit = 's';
    data{i}.InputUnit = 'N';
    data{i}.OutputUnit = 'm';
end
data_sim = iddata(delta_z_sim.diff_trajet(:,1),...
    delta_fz_sim.diff_trajet(:,1),dt); % ideal data
data_sim.TimeUnit = 's';

```

```
data_sim.InputUnit = 'N';
data_sim.OutputUnit = 'm';
```

The ARX identification method can now be used, since we know the order of the polynomials A_k and B_k :

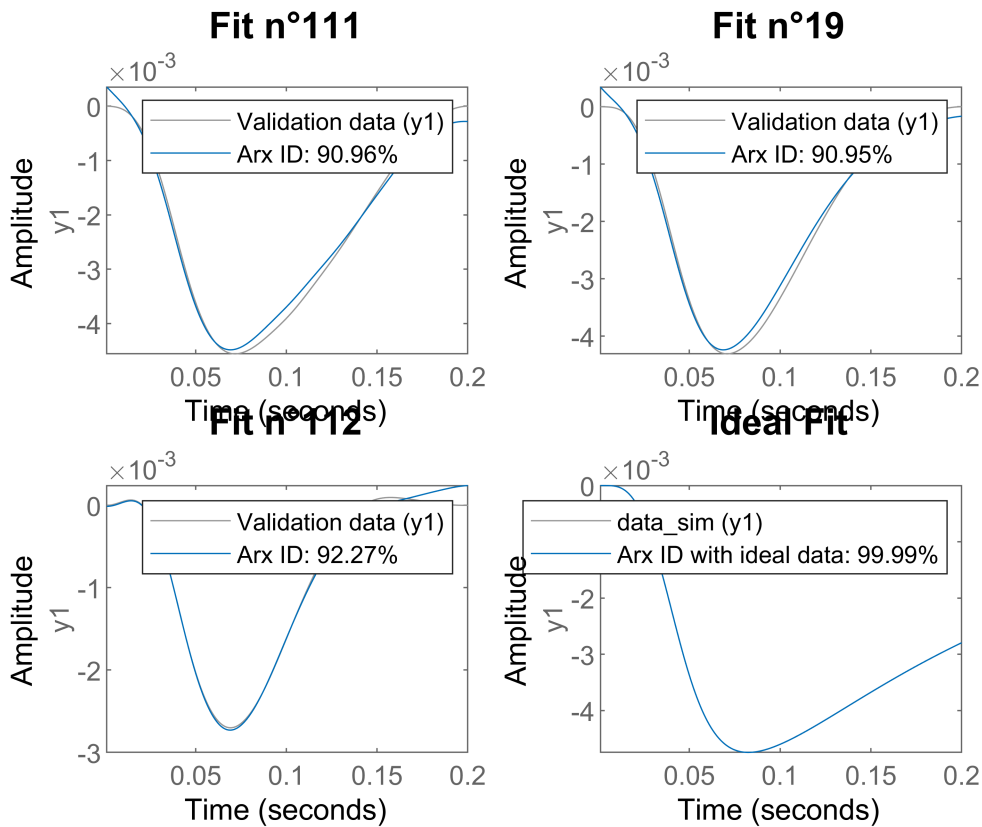
```
na = length(a_coef)-1; % order of Ak
nb = length(b_coef)-1; % order of Bk
nk = 0; % delay in Bk
for i = delta_z.nb_traject:-1:1
    arx_id{i} = arx(data{i},[na nb+1 nk], 'IntegrateNoise',true);
    arx_id{i}.Name = 'Arx ID';
end
arx_id_sim = arx(data_sim,[na nb+1 nk]);
arx_id_sim.Name = 'Arx ID with ideal data';
```

The quality of the identification can be estimated... to be continued...

```
%
% for i = delta_z.nb_traject:-1:1
%     pem_id{i} = pem(data{i},arx_id{i});
%     pem_id{i}.Name = 'Pem ID';
% end
```

A random estimation is picked to compare the quality of the estimation:

```
rand_idx = randi([1 delta_z.nb_traject],3,1);
figure('DefaultAxesFontSize',14)
for i = 1:3
    subplot(2,2,i)
    compare(data{rand_idx(i)},arx_id{rand_idx(i)});
    title("Fit n°" + string(rand_idx(i)))
end
%figure('DefaultAxesFontSize',14)
subplot(2,2,4)
compare(data_sim,arx_id_sim);
title("Ideal Fit")
```

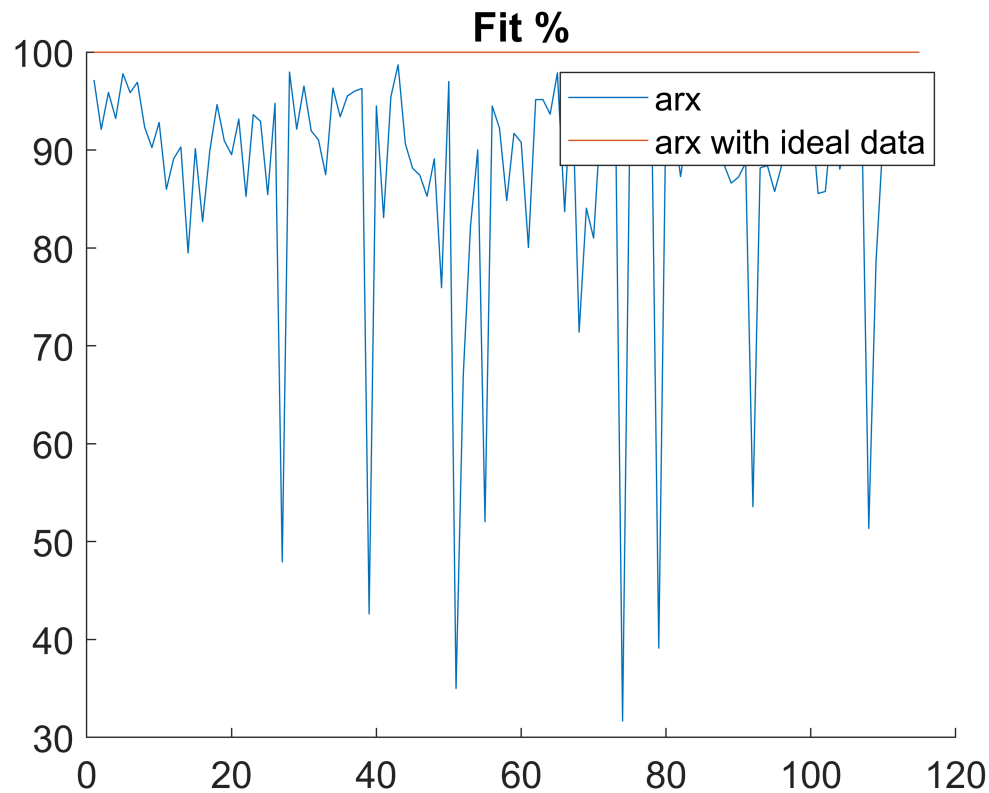


A wider overview is then displayed:

```

for i = delta_z.nb_traject:-1:1
    % Report fit default horizon is set to 1, how do we change it ?
    %arx_id_fit1(i) = arx_id{i}.Report.Fit.FitPercent;
    [~,arx_id_fit(i),~] = compare(data{i},arx_id{i});
end
[~,arx_id_sim_fit,~] = compare(data_sim,arx_id_sim);
figure('DefaultAxesFontSize',14)
hold on
plot(arx_id_fit)
plot([1, delta_z.nb_traject], [arx_id_sim_fit, arx_id_sim_fit])
title('Fit %')
legend('arx', 'arx with ideal data')

```



Impedance parameters identification

Once the coefficients of A_k and B_k are identified, the impedance parameters can be deduced thanks to the relations detailed in the *Discretization* paragraph for every identification conducted for each perturbation:

$$a_1 = \frac{a_1'}{a_2'}, a_0 = \frac{a_0'}{a_2'}, K_h(1 + b_0) = \frac{b_1' + b_0'}{a_2'}$$

```
% K D M identification
for i = 1:length(arx_id)
    % declare the equations system
    eq.A(1) = a1 - arx_id{i}.A(2)/arx_id{i}.A(1) == 0;
    eq.A(2) = a0 - arx_id{i}.A(3)/arx_id{i}.A(1) == 0;
    eq.B(1) = Kh*(b0 + 1) - (arx_id{i}.B(1) + arx_id{i}.B(2))/arx_id{i}.A(1) == 0;
    imp_param_id(i) = solve({eq.A(1),eq.A(2),eq.B(1)},[Ks;Bs;Ms]);
    if i == 1
        warning('off','all'); % to avoid spam warnings in the loop
    end
end
```

Warning: Unable to solve symbolically. Returning a numeric solution using vpsolve.

```
warning('on','all')
```

Same process on the ideal case scenario

```
eq.A(1) = a1 - arx_id_sim.A(2)/arx_id_sim.A(1) == 0;  
eq.A(2) = a0 - arx_id_sim.A(3)/arx_id_sim.A(1) == 0;  
eq.B(1) = Kh - arx_id_sim.B(1)/arx_id_sim.A(1) == 0;  
eq.B(2) = b0 - arx_id_sim.B(2)/arx_id_sim.B(1) == 0;  
eq.B(3) = Kh*b0 - arx_id_sim.B(2)/arx_id_sim.A(1) == 0;  
eq.B(4) = Kh*(b0 + 1) - (arx_id_sim.B(1) + arx_id_sim.B(2))/arx_id_sim.A(1) == 0;  
imp_param_id_sim = solve({eq.A(1),eq.A(2),eq.B(4)},[Ks;Bs;Ms]);
```

Warning: Unable to solve symbolically. Returning a numeric solution using vpasolve.

```
% expected arx param  
double(subs(a1, [Ks, Bs, Ms], [Kv, Bv, Mv]))
```

```
ans = -1.9510
```

```
double(subs(a0, [Ks, Bs, Ms], [Kv, Bv, Mv]))
```

```
ans = 0.9512
```

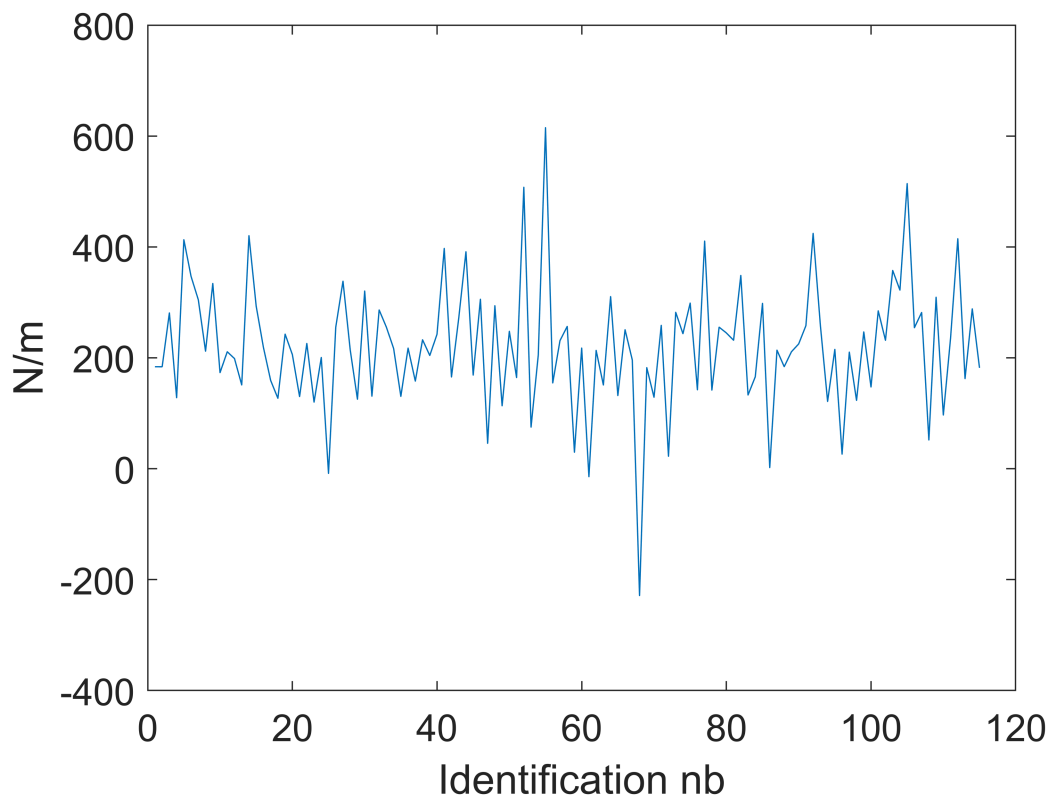
```
double(subs(Kh*b0, [Ks, Bs, Ms], [Kv, Bv, Mv]))
```

```
ans = 2.4182e-06
```

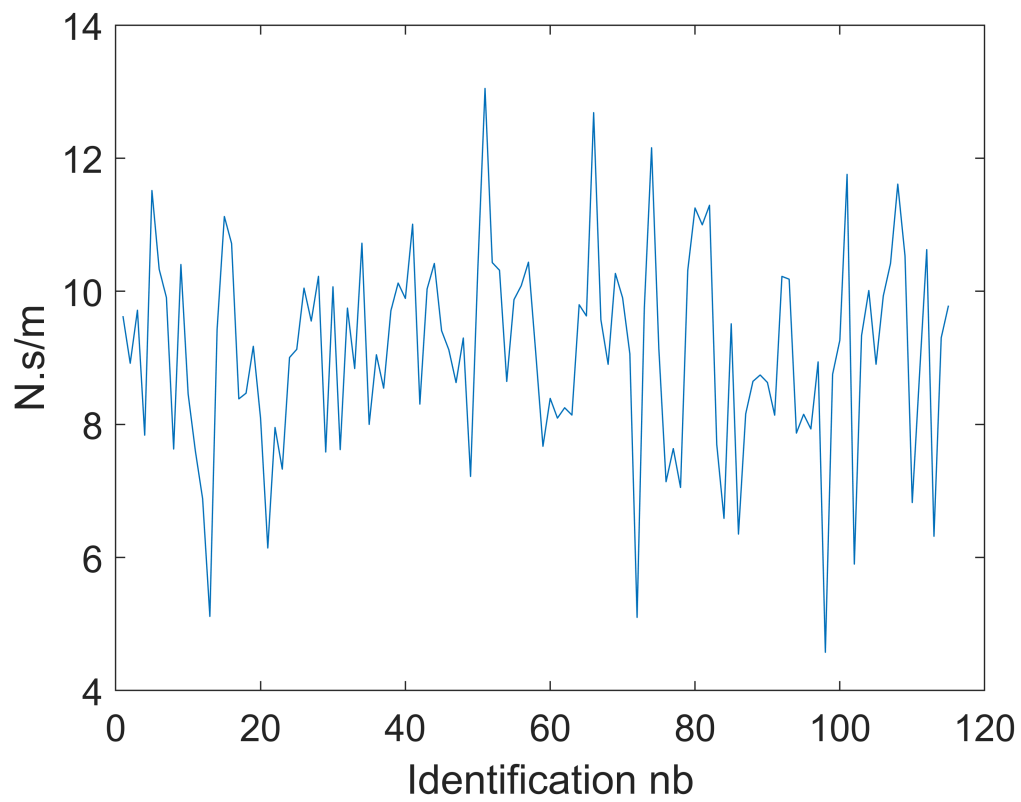
Results

The identified parameters are:

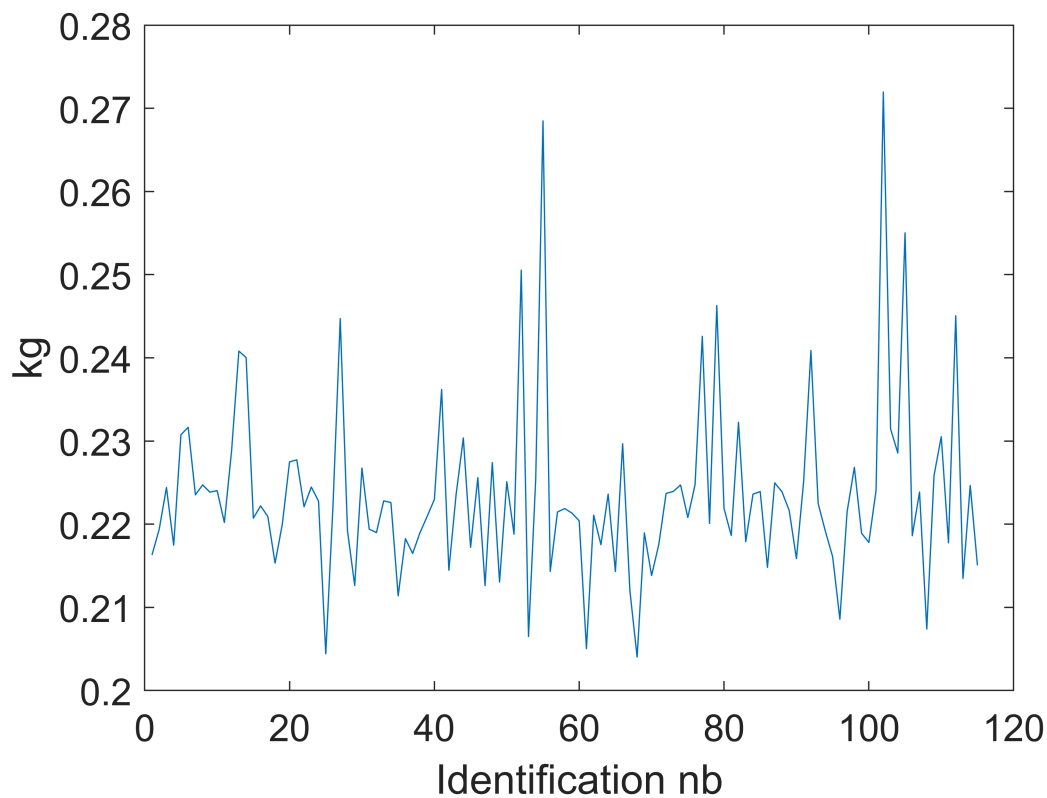
```
figure('DefaultAxesFontSize',14)  
title('Stiffness')  
plot([imp_param_id(:).Ks])  
xlabel('Identification nb')  
ylabel('N/m')
```



```
figure('DefaultAxesFontSize',14)
title('Damping')
plot([imp_param_id(:).Bs])
xlabel('Identification nb')
ylabel('N.s/m')
```



```
figure('DefaultAxesFontSize',14)
title('Mass')
plot([imp_param_id(:).Ms])
xlabel('Identification nb')
ylabel('kg')
```



If we take away the outliers, and compute the average and the standard deviation, we have the following results:

```
mean(rmoutliers(double([imp_param_id(:).Ks])))
```

```
ans = 215.2212
```

```
mean(rmoutliers(double([imp_param_id(:).Bs])))
```

```
ans = 9.1250
```

```
mean(rmoutliers(double([imp_param_id(:).Ms])))
```

```
ans = 0.2212
```

In the ideal case scenario, where the virtual trajectories are known, the methodology gives the following relative error to the 3 impedance parameters:

```
abs(double(imp_param_id_sim.Ks)-Kv)/Kv*100
```

```
ans = 3.8450e-04
```

```
abs(double(imp_param_id_sim.Bs)-Bv)/Bv*100
```

```
ans = 6.3613e-05
```

```
abs(double(imp_param_id_sim.Ms)-Mv)/Mv*100
```

```
ans = 8.9561e-04
```

Annexes

Functions

```
function F = root2d(X, eq)
syms Ks Bs Ms
F(1) = subs(eq.A(1), [Ks, Bs, Ms], [X(1), X(2), X(3)]);
F(2) = subs(eq.A(2), [Ks, Bs, Ms], [X(1), X(2), X(3)]);
F(3) = subs(eq.B(2), [Ks, Bs, Ms], [X(1), X(2), X(3)]);
end

function b = copyObj(a)
    b = eval(class(a)); %create default object of the same class as a. one valid use of eval
    for p = properties(a).' %copy all public properties
        try %may fail if property is read-only
            b.(p{1}) = a.(p{1});
        catch
            warning('failed to copy property: %s', p);
        end
    end
end
```